Ders 1 Genişletilmiş Notu: Asimptotik Notasyonlar ve Özyinelemeli Algoritmaların Analizine Giriş

Ders 1 Genişletilmiş Notu: Asimptotik Notasyonlar ve Özyinelemeli Algoritmaların Analizine Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Haftanın Tekrarı ve Hesaplama Modeli
- 2. İki Algoritmayı Karşılaştırma Sorunu
- 3. Çalışma Zamanı Fonksiyonu
- 4. Big O Notasyonu: Üst Sınır
- 5. Omega Notasyonu: Alt Sınır
- 6. Theta Notasyonu: Sıkı Sınır
- 7. Little o Notasyonu ve Kurallar
- 8. Yaygın Fonksiyonlar ve Limit ile Karşılaştırma
- 9. Maksimum Alt Dizi Toplamı Problemi
- 10. Döngülerin ve if-else Yapısının Analizi
- 11. En Kötü, Ortalama ve En İyi Durum
- 12. Hazır Algoritma Seçimi ve Analizin Deneysel Kontrolü
- 13. Algoritma Karmaşıklığı ve Problem Karmaşıklığı
- 14. Özyinelemeli Çağrılar ve Yineleme Bağıntıları
- 15. Hanoi Kuleleri ve Kök İçeren Bir Yineleme
- Örnek Sorular

1. Geçen Haftanın Tekrarı ve Hesaplama Modeli

Ders, önceki haftanın kısa bir tekrarıyla başladı. Geçen hafta bu derste kullanılacak temel veri yapıları hızlıca gözden geçirilmiş, sonra algoritmanın ne olduğu ve özellikleri konuşulmuştu. Bu özellikler şunlardır:

- **Açıklık (clear):** Algoritmanın bir adımı söylendiğinde herkes aynı şeyi anlamalıdır.
- **Sonluluk (finite):** Algoritma belli bir süre sonra mutlaka sonlanmalıdır.
- **Etkinlik (effective):** Çok verimsiz bir yöntemi "bu da bir algoritmadır" diye sunmanın anlamı yoktur, çünkü bir noktadan sonra kullanılamaz.

Ders "algoritma analizi ve tasarımı" adını taşıdığı için bu hafta analiz tarafından devam edildi. Geçen hafta bir dizinin elemanlarını toplayan basit bir kodun çalışma zamanı table yöntemiyle (her satırın kaç kez çalıştığını sayarak) bulunmuştu. Bu hafta konu bir adım geriden alındı.

Bir bilgisayarın çalışmasını analiz etmek için farklı modeller vardır: otomatlar, Turing makineleri ya da RAM modeli. Derste bugün kullandığımız bilgisayarlara benzeyen RAM modeli kullanılır. Bu modelde toplama, çarpma, bölme, atama gibi bütün temel işlemlerin birim zaman (unit time) aldığı, yani eşit sürdüğü varsayılır. Her kelimenin (word), kabaca her değişkenin, örneğin 32 bit gibi aynı yeri kapladığı düşünülür.

Hoca bilgisayar mimarisini hatırlattı: gerçek bir işlemcide bazı komutlar tek adımda yapılırken, bazıları tek bir komut olduğu hâlde CPU, bellek ve register döngüsünde çok daha fazla adım

sürer. Analizde böyle karmaşık işlemler olmadığı varsayılır. Karmaşık bir işleme gerçekten ihtiyaç varsa (örneğin kaydırıp tekrar toplayan bir işlem), bu işlem algoritmanın içinde açıkça yazılır ve içindeki toplama, çıkarma gibi temel işlemlerden süresi hesaplanır.

2. İki Algoritmayı Karşılaştırma Sorunu

Slaytta iki algoritmanın çalışma zamanlarının bulunduğu varsayılıyor (ekrandan, 00:05:20):

- A algoritmasının çalışma zamanı: $T_A(N) = 1000N$
- B algoritmasının çalışma zamanı: $T_B(N) = N^2$

Soru "hangisi daha hızlı?" idi. Sohbet kutusundan $1000N$ cevabı geldi, gerekçe olarak da "doğrusal olduğu için" denildi. Hoca bu cevabı çürütmeye çalıştı. Birim mikrosaniye olsun:

N	$T_A = 1000N$	$T_B = N^2$	Daha hızlı
10	10^{-2} s	10^{-4} s	B
100	10^{-1} s	10^{-2} s	B
1000	1 s	1 s	eşit
10000	10 s	100 s	A

Girdi boyutu 1000'den küçükken B daha hızlıdır, 1000'de ikisi eşittir, 1000'den sonra B'nin süresi çok daha hızlı artar. Dolayısıyla doğru cevap, girdi boyutu bilinmeden "A daha hızlı" ya da "B daha hızlı" diye kesin bir şey söylenemeyeceğidir. Grafikte x ekseni girdi boyutu, y ekseni süredir; 1000'e kadar B'nin eğrisi altta kalır, sonra A'nınkini geçip çok daha yukarı çıkar.

Bu fonksiyonların ortak özelliği artan olmalarıdır. Bir algoritmanın çalışma zamanında girdi arttıkça süre ya sabit kalır ya da artar; hiçbir zaman azalmaz. Bu tür fonksiyonlara azalmayan fonksiyon (non-decreasing function) denir.

T_A ve T_B'yi kesin bir şekilde karşılaştıramadığımız için asimptotik notasyon adı verilen matematiksel yöntem kullanılır. Notasyon, bir şeyi göstermek için kullanılan simgedir; bir yerde sigma, başka bir yerde lambda yazarsınız ve o simge bir anlam taşır. Asimptotik notasyonlar da algoritmaların çalışma zamanlarını yine bir fonksiyonla göstermeye yarar.

Slayttaki soru "kesin sonuç almak her zaman mümkün mü?" ve cevabı "hayır" (ekrandan, 00:10:00). Çalışma zamanı karmaşıklığı (run-time complexity), algoritmayı gerçekten makinede çalıştırınca ölçülen süredir ve şunlardan etkilenir:

- Derleyici, işletim sistemi ve bilgisayar: bunlar genellikle sabitleri ve düşük dereceli terimleri etkiler.
- Algoritmanın kendisi: bunun için asimptotik notasyon kullanılır.

Bir bilgisayarda A hızlı çıkarken başka bir bilgisayarda B hızlı çıkabilir. Asimptotik notasyonla algoritmalar derleyiciden, işletim sisteminden ve donanımdan bağımsız olarak karşılaştırılır.

3. Çalışma Zamanı Fonksiyonu

Çalışma zamanı fonksiyonu (running time function), azalmayan bir fonksiyondur. Pozitif tam sayılardan ($\mathbb{Z}^+$) gerçel sayılara ($\mathbb{R}$) gider ve belli bir m'den büyük her n için $f(n) > 0$'dır. Hoca bunu şöyle açıkladı: fonksiyonun girdisi input büyüklüğüdür, bu yüzden tam sayıdır; çıktısı gerçel bir değerdir. Girdi büyüklüğü negatif olamayacağı için fonksiyonun değeri sıfırdan büyüktür. Bundan sonraki notasyonlarda geçen $T(n)$ ve $f(n)$ hep bu tür fonksiyonlardır.

4. Big O Notasyonu: Üst Sınır

4.1 Tanım

Slayttaki tanım şöyledir (ekrandan, 00:13:00):

> $T(N)$ ve $f(N)$ çalışma zamanı fonksiyonları olsun. Pozitif c ve n_0 sabitleri için $N \geq n_0$ iken $T(N) \leq c\, f(N)$ oluyorsa $T(N) = O(f(N))$ yazılır.

Bunun anlamları slaytta şöyle sıralanmış:

- $T(N)$, $f(N)$'den daha hızlı büyümez.
- Büyük N için $T(N)$'nin büyüme hızı $f(N)$'ninkinden küçük ya da ona eşittir.
- $f(N)$, $T(N)$ için bir üst sınırdır.

Hoca tanımı bir grafikle anlattı. x ekseninde n, y ekseninde çalışma zamanı vardır. n_0 noktasından sonra $c\,f(n)$ eğrisi (yeşil) $T(n)$ eğrisinin (mavi) hep üstünde kalır. n_0'dan küçük değerler bizi ilgilendirmez; orada mavi eğri yeşilin üstünde de olabilir. Önemli olan bir değerden sonra $T(n)$'nin daha az artmasıdır.

İlk örnek $1000n$ idi: $c = 2000$ ve $n_0 = 1$ seçilirse $1000n \leq 2000n$ olur, dolayısıyla $T_A(n) = 1000n = O(n)$.

4.2 Örnekler

Slayttaki örnekler (ekrandan, 00:20:40):

$7n{+}5 = O(n)$ **mi?** $7n{+}5 \leq c\,n$ eşitsizliğini sağlayan sabitler aranır. $c = 8$ seçilirse $7n{+}5 \leq 8n$ eşitsizliği $n \geq 5$ için sağlanır. Örneğin $n = 10$'da $75 \leq 80$. Konulabilecek en küçük n değeri 5'tir ($40 \leq 40$). Dolayısıyla $7n + 5 = O(n)$.

$7n + 5 = O(n^2)$ **mi?** $7n + 5 \leq c\,n^2$ için $c = 7$ ve $n_0 = 2$ seçilir. $n = 2$'de $19 \leq 28$ olur ve 3, 4, 5, ... için de sağlanır. Dolayısıyla $7n + 5 = O(n^2)$ da doğrudur.

$7n^2 + 3n = O(n)$ **mi?** $7n^2 + 3n \leq c\,n$ eşitsizliğini sağlayan c ve n_0 aranır. n sıfır olamaz, çünkü girdi pozitif tam sayıdır; birden başlanır. $n = 1$'de $10 \leq c$ çıkar, yani $c = 10$ alınabilir. Ama c sabitken bundan sonraki bütün n'ler için de sağlanması gerekir. $n = 2$'de sol taraf 34, sağ taraf $10 \cdot 2 = 20$ olur ve eşitsizlik sağlanmaz. Bu şekilde sabit bir c bulunamaz, dolayısıyla $7n^2 + 3n \neq O(n)$. Grafikte de $7n + 5$ ile n birbirine yakın giderken n^2'li terim n büyüdükçe çok daha hızlı artar.

4.3 Üst sınırın anlamı

Bir öğrencinin "$B = O(A)$ yazınca tam olarak ne kastediliyor?" sorusu üzerine hoca örneği yeniden kurdu: $A = 1000n$, $B = n^2$ olsun. $1000n \leq c \cdot n \cdot n$ eşitsizliği yazılabilir mi? $c = 1$ alınırsa $n \geq 1000$ için sağlanır; yani $n_0 = 1000$ seçilerek $1000n = O(n^2)$ yazılabilir. Hoca bu sırada c ile n_0'ı karıştırmamak gerektiğini, c'nin fonksiyonu yukarı aşağı kaydırdığını, n_0'ın ise eşitsizliğin geçerli olmaya başladığı noktayı gösterdiğini vurguladı. 1000'den önce A'nın süresi daha büyüktür, 1000'den sonra B çok daha hızlı büyür. "$1000n = O(n^2)$" demek, belli bir değerden sonra n^2'nin $1000n$'den çok daha hızlı büyüdüğü, yani onun üst sınırı olduğu demektir.

Katsayılar ve düşük dereceli terimler için iki gözlem yapıldı:

- Üsleri aynı terimlerde öndeki katsayı önemli değildir. $7n$ ile n arasındaki fark, tanımdaki c katsayısıyla karşılanır.
- Polinomlarda düşük dereceli terimlerin önemi azalır. n^3 bir yerden sonra n^2'den çok daha hızlı büyüdüğü için $n^3 + 7n^2 + 3$ gibi bir ifade $O(n^3)$ olarak yazılır.

Slaytta bu iki algoritma $T_A(N) = 1000N = O(N)$ ve $T_B(N) = N^2 = O(N^2)$ olarak yazılmış ve "A, B'den asimptotik olarak daha hızlıdır" denmiştir (ekrandan, 00:40:00).

4.4 Sıkı üst sınır

$1000n = O(n)$ ise $1000n = O(n^2)$ yazılabilir mi? Bir öğrenci önceki slaytlarda yazılabildiğini hatırlattı. Gerekçe tanımdan gelir: $1000n \leq c\,n$ ise $c\,n \leq c\,n^2 \leq c\,n^3 \leq c\,n^4 \leq \ldots$ olur. Dolayısıyla $1000n$ için $O(n^2)$, $O(n^3)$, $O(n^4)$ de doğrudur.

Hoca burada öğrencilerin şu itirazı yapması gerektiğini söyledi: "O zaman n^∞ yazsam da Big O'ya uyar." Evet, bunların hepsi üst sınırdır, ama açıklayıcı değildir. Aranan şey daha sıkı (tight) bir üst sınırdır. Hoca bunu bir benzetmeyle anlattı: "bu olay beş yıl içinde olacak" ile "bu olay

bir ay içinde olacak" sözlerinden ikincisi daha kesindir, çünkü sınırı daha sıkıdır. $1000n = O(n^2)$ demek yanlış değildir ama sıkı bir sınır olmadığı için işe yaramaz.

4.5 Neden asimptotik notasyon kullanıyoruz?

Table yöntemiyle büyük bir algoritma analiz edilince elde büyük bir polinom kalır. İç içe döngüler, n^2 ve $(n+1)^2$ kez dönen satırlar derken çalışma zamanı örneğin $4n^3 + 2n^2 + 189$ gibi çıkar; slaytta hoca bunu $4n^3 + 2n \cdot n + 189 = O(n^3)$ diye yazdı (ekrandan, 00:40:00). Başka birinin yazdığı kod da başka bir çok terimli ifade verir. Bu polinomları doğrudan karşılaştırmak anlamlı değildir. Bunun yerine algoritmaların üst sınırları kabaca verilir: öndeki katsayılar ve düşük dereceli terimler atılır.

"Bu algoritma $O(n^3)$ çalışır" demek, $4n^3 + \ldots$ süreli algoritmanın n^3'ten daha hızlı büyümediği, yani n^3 süreli bir algoritmanın altında kaldığı demektir. Gerçek analizde if'ler yüzünden if'e girme olasılıkları da hesaba katılır, bazen logaritmalı ve başka terimler çıkar. Asimptotik notasyon bu terimlerden kurtulup algoritmanın süresini kabaca göstermeyi sağlar. Big O algoritmanın üst sınırını verir; algoritma en kötü durumda bu kadar yavaş çalışır.

Ardından gelen sorular:

Soru: Birden fazla girdi varsa yine tek bir n mi gösteriyoruz? **Cevap:** Genellikle girdiler birleştirilir. Fonksiyon iki dizi alıyorsa, biri 1000 diğeri 2000 elemanlıysa, n 3000 olarak alınır.

Soru: n^4, n^3, n^2 terimleri varsa en yüksek kuvveti mi alıyoruz? **Cevap:** Evet, en yüksek dereceli terim alınır.

Soru: $4n^3$'ü $O(n^3)$ yaparken $c = 5$ mi demeliyiz? **Cevap:** Polinomda başka terim yoksa $c = 4$ denebilir, çünkü $4n^3 \leq 4n^3$ olur; $3n^3$ ise yetmez. Başka terimler varsa fark eder, çünkü c bazı polinomlarda n^2'li terimlerden gelen kısmı da karşılamak zorunda kalabilir. Hoca c'yi sezgisel olarak şöyle anlattı: bir fonksiyonu bir sabitle çarpmak grafiğini biraz kaydırmak demektir; c ile polinomun yerini biraz değiştirirsiniz.

5. Omega Notasyonu: Alt Sınır

5.1 Tanım

Big O'da fonksiyonun üstünde kalan bir fonksiyon söyleniyordu. Omega notasyonu bunun tersidir: alt sınırı söyler. Alt sınır, algoritmanın bu sınırdan daha hızlı çalışmayacağı anlamına gelir.

Hoca grafikte kırmızı eğriyi kendi algoritması, yeşil eğriyi onun Big O ile gösterilen üst sınırı (n^2 ya da n^3) olarak çizdi. Omega, algoritmanın altında kalan bir eğridir (örneğin n); algoritma bundan daha aşağıda, yani daha hızlı çalışmaz.

$T(n) = \Omega(f(n))$ ancak ve ancak pozitif c ve n_0 için $n \geq n_0$ iken $T(n) \geq c\, f(n)$ ise.

5.2 Örnekler

Slayttaki örnekler (ekrandan, 00:50:00):

- $3n + 2 = \Omega(n)$**:** $3n + 2 \geq c\, n$ yazılabiliyorsa doğrudur. Burada da düşük dereceli terimler ve katsayılar atılabilir. Hoca slayta $3n + 2$ için $= O(n)$ notunu da ekledi: aynı ifade hem $O(n)$ hem $\Omega(n)$'dir; buna birazdan dönüldü.

  Ders sonrası not: Hoca bu örnekte "$n > 2$ için, $c = 4$" dedi. $3n + 2 \geq 4n$ eşitsizliği yalnız $n \leq 2$ için doğrudur. Doğru seçim $c = 3$ (ya da 3'ten küçük herhangi bir pozitif sabit) ve $n_0 = 1$'dir: her $n \geq 1$ için $3n + 2 \geq 3n$.

- $6 \cdot 2^n + n^2 = \Omega(2^n)$**:** $6 \cdot 2^n + n^2 \geq c \cdot 2^n$ eşitsizliğini sağlayan değerler bulunursa yazılabilir.
- $3n - 7 = \Omega(1)$**:** Hoca bunu "absürt bir durum" olarak gösterdi. $3n - 7$ doğrusal bir fonksiyondur; 1 ise sabit bir fonksiyondur. İfade doğrudur, çünkü sabit fonksiyon bunun alt sınırıdır. Ama iyi bir alt sınır değildir, çünkü 1 zaten çoğu şeyin alt sınırıdır. Üst sınır ararken fonksiyona yakın üst sınırlar arandığı gibi, alt sınır ararken de fonksiyona yakın, sıkı alt sınırlar aranır.

5.3 Big O ile Omega arasındaki ilişki

$n^2 = O(n^3)$ doğrudur, çünkü n^3 daha hızlı büyür. Aynı ilişki $n^3 = \Omega(n^2)$ biçiminde de yazılabilir: n^3, n^2'nin üst sınırıysa n^2 de n^3'ün alt sınırıdır. Bu tanımlardan çıkar.

$f(n) = 2n^2$ ve $g(n) = n^2$ olsun. $f = O(g)$ mi? Evet, çünkü $c = 2$ için $2n^2 \leq c\,n^2$. $f = \Omega(g)$ mi? Evet, çünkü yine $c = 2$ için $2n^2 \geq c\,n^2$. Yani hem Big O hem Omega doğrudur ve ikisi de sıkı sınırdır.

6. Theta Notasyonu: Sıkı Sınır

6.1 Tanım ve sezgi

Bir fonksiyon aynı fonksiyonla hem Big O hem Omega olarak yazılabiliyorsa, üst sınır ile alt sınır neredeyse aynı demektir. "Neredeyse" denmesinin sebebi düşük dereceli terimlerin atılmasıdır. Bu durumda Theta notasyonu kullanılır:

$$T(n) = O(h(n)) \text{ ve } T(n) = \Omega(h(n)) \text{ ise } T(n) = \Theta(h(n)).$$

Theta, üst sınır ile alt sınırın sıkı olduğu, fonksiyona yaklaşık olduğu durumu ifade eder. f, g ile aynı oranda büyür; hem üst hem alt sınırı olan şey aslında fonksiyonun kendisidir denebilir.

Slaytta $f(n) = O(g(n))$ ancak ve ancak $g(n) = \Omega(f(n))$ olduğu da gösterilmiş. Big O tanımından $f(n) \leq c_1 g(n)$ yazılır. İki taraf c_1'e bölününce $\frac{1}{c_1} f(n) \leq g(n)$ elde edilir. $c_2 = \frac{1}{c_1}$ seçilirse $g(n) \geq c_2 f(n)$ olur; bu da Omega tanımıdır.

6.2 Örnek

Slayttaki örnek (ekrandan, 01:00:00):

- $7n^2 + 3n + 5 = O(n^4)$
- $7n^2 + 3n + 5 = O(n^3)$
- $7n^2 + 3n + 5 = O(n^2)$
- $7n^2 + 3n + 5 = \Omega(n^2)$
- $7n^2 + 3n + 5 = \Omega(n)$
- $7n^2 + 3n + 5 = \Omega(1)$

Sonuç: $7n^2 + 3n + 5 = O(n^2) = \Omega(n^2) = \Theta(n^2)$.

Düşük dereceli $3n + 5$ ve katsayı 7 atılınca ifade $O(n^2)$ olur; n^3 ve n^4 de üst sınırdır. Omega olarak n^2, n ve 1'in hepsi alt sınırdır. Hoca bunu grafikte gösterdi: n^2 eğrisi doğrusal değil, hızla artan bir eğridir; n^2, n^3, n^4 bunun üst sınırları, n^2, n ve düz bir çizgi olan 1 alt sınırlarıdır. Ama gereken sıkı olanlardır, çünkü $\Omega(1)$ neredeyse her şeyin altındadır. Bir ifade n^2 ile hem Big O hem Omega olarak yazılabiliyorsa ona Theta denir.

Biri "algoritmamın çalışma zamanı $\Theta(n^2)$" diyorsa, hem üst hem alt sınırın n^2 olduğu bilinir. Bu yüzden notasyonlar içinde algoritma hakkında en faydalı bilgiyi Theta verir.

Soru: $\Omega(n)$ nasıl yazılabildi? **Cevap:** $7n^2 + 3n + 5 \geq c \cdot n$ sağlayan bir değer bulunabilir mi diye bakılır. $c = 8$ alınsın. $n = 1$'de $7 + 3 + 5 = 15 \geq 8$. $n = 2$'de ve $n = 3$'te de sağlanır, çünkü sol taraf çok daha hızlı büyür. Dolayısıyla $\Omega(n)$ yazılabilir.

Soru: Theta'yı tespit edemediğimiz durumlar olabilir mi? **Cevap:** Evet. Polinom çok karışık olduğunda Theta tespit edilemeyebilir; o zaman Big O ve Omega kullanılmak zorunda kalınır, bazen sıkı olmayan bir alt sınırla yetinilir. Bu yüzden diğer notasyonlara da gerek vardır. Grafikte n^2 deyince düzgün artan bir eğri akla gelir, ama gerçek polinom bazen hızlı, bazen yavaş artar; artar, biraz düz gider, tekrar artar. Big O'nun avantajı bu ayrıntıları yok saymasıdır, ama her zaman hem altta hem üstte aynı fonksiyonla sıkıştırmak mümkün olmaz.

Soru: Theta, Omega ile Big O'nun eşit olduğu durumda var diyebiliyoruz değil mi? **Cevap:** Evet.

Soru: Koşul deyimi çok olan bir algoritmada Theta bulmak zor olur diyebilir miyiz? **Cevap:** Her zaman denemez. Koşullarda if'e girme olasılığı hesaplanır, yani her şey olasılıklar üzerinden yürür. Koşul varsa Theta bulmanın daha zor olabileceği yuvarlak bir ifade olarak söylenebilir. Koşula girip girmemek genellikle girdinin hangi dağılımdan üretildiğine bağlıdır; bu yüzden problemden probleme çok değişir ve her problem kendi özelinde değerlendirilmelidir. Bu dersteki algoritmaların çoğunda, içlerinde if olsa da, Theta bulunabilecektir.

Aradan sonra hoca Theta örnekleriyle devam etti: $3n + 2 = \Theta(n)$ ve $10 \log n + 4 = \Theta(\log n)$ yazılabilir, çünkü $\log n$ hem alt hem üst sınır olarak yazılabilir. $3n + 2 = \Theta(1)$ yazılamaz: 1 bunun alt sınırıdır ($\Omega(1)$), ama üst sınır olamaz. $3n^2$ için $O(n^4)$ ya da $O(n^2)$ denebilir, ama $\Theta(n^2)$ demek hem üst hem alt sınırın n^2 olduğunu söylediği için daha sıkı bir bilgidir.

7. Little o Notasyonu ve Kurallar

7.1 Little o

Big O tanımında "küçük eşittir" vardı. Eşitlik kaldırılıp "küçüktür" konursa little o elde edilir. Başka bir ifadeyle little o yine bir üst sınırdır, ama bu üst sınır asla sıkı sınır değildir:

$T(n) = O(p(n))$ ve $T(n) \neq \Theta(p(n))$ ise $T(n) = o(p(n))$.

Örnek: $T(n) = 3n^2$ olsun. $3n^2 = \Theta(n^2)$'dir, çünkü hem Big O hem Omega olarak n^2 ile yazılabilir. Buna göre $3n^2 = o(n^4)$ ve $3n^2 = o(n^3)$ doğrudur. $3n^2 = o(n^2)$ ise yanlıştır, çünkü n^2 Theta sınırıdır. Little o, Big O'nun Theta olmayan durumlarıdır: gerçek çalışma zamanına eşit olmayan, daha yukarıdaki üst sınırları gösterir.

7.2 Kural 1

Slayttaki kural (ekrandan, 01:18:10):

$T_1(N) = O(f(N))$ ve $T_2(N) = O(g(N))$ ise a) $T_1(N)+T_2(N) = \max(O(f(N)), O(g(N)))$
b) $T_1(N) \cdot T_2(N) = O(f(N) \cdot g(N))$

Slaytta ayrıca "Bunları ispatlayabilir misiniz? Theta için doğru mu? Omega için ne olur?" soruları var. Hoca ispatın fikrini verdi: T_1 ve T_2'yi en yüksek dereceleri f ve g olan polinomlar gibi düşünün. İki polinom toplanırken aynı dereceli terimlerin katsayıları toplanır; bu yüzden toplamın Big O'su değişmez, hangisinin derecesi büyükse ona eşit olur. İki polinom çarpılırken üsler toplanır; bu yüzden sonuç çarpımın mertebesindedir. Örneğin $f\ n^2$, $g\ n^3$ olabilir.

7.3 Kural 2

Slayttaki kural (ekrandan, 01:20:00):

$T(N)$ k. dereceden bir polinomsa, yani $T(N) = a_k N^k + a_{k-1} N^{k-1} + \cdots + a_1 N + a_0$ ise $T(N) = \Theta(N^k)$.

Bu, Big O için daha önce söylenen "düşük dereceli terimler atılabilir" gözleminin Theta biçimidir. Theta yazılabilmesinin sebebi, polinomun hem küçük eşit hem büyük eşit ile N^k cinsinden yazılabilmesidir. Slayttaki basit ispatta küçük eşit tarafındaki terim istenildiği kadar büyütülebildiği için bütün katsayılar en büyük katsayı a ile değiştirilir ve sonra parantez dışına N'nin kuvveti alınır.

7.4 Logaritma

Şimdiye kadar hep n'lerle konuşuldu. Logaritma çok daha yavaş büyür. Her k için

$$\log^k n = o(n)$$

yazılır: logaritmanın kuvveti ne olursa olsun n daha hızlı büyür.

Soru: Oradaki k taban mı, üs mü? **Cevap:** Üs. Taban hep 2 gibi düşünülür. $k = 2$ ise $\log^2 n = \log n \cdot \log n$ demektir.

7.5 Theta bir denklik bağıntısıdır

Slaytta Theta'nın bir denklik bağıntısı olduğu söylenir. $f(n) = \Theta(f(n))$ her zaman sağlanır, çünkü hem küçük eşit hem büyük eşit yazılabilir. $f(n) = \Theta(g(n))$ ise $g(n) = \Theta(f(n))$ de yazılabilir. Hoca bunların hepsinin $f(n) \leq c\, g(n)$ biçimindeki tanım kullanılarak rahatlıkla ispatlanabileceğini söyledi.

8. Yaygın Fonksiyonlar ve Limit ile Karşılaştırma

8.1 Yaygın fonksiyonlar

Slayttaki liste (ekrandan, 01:23:30):

- $c = o(\log N)$; yani $c = O(\log N)$ ama $c \neq \Omega(\log N)$
- $\log N = o(\log^2 N)$
- $\log^2 N = o(N)$
- $N = o(N \log N)$
- $N = o(N^2)$
- $N^2 = o(N^3)$
- $N^3 = o(2^N)$

Her satır, soldaki fonksiyonun sağdakinden daha yavaş büyüdüğünü söyler. Bu fonksiyonlar çizilirse en hızlı büyüyen 2^N'dir, yani en yavaş çalışan algoritma budur. Ardından polinomlar, sonra logaritmalı terimler gelir, en yavaş büyüyen de sabittir. Çalışma zamanı en iyi sabitken, en kötü 2^N'dir.

8.2 Stil

$T(N) = 4N^2$ ise bu hemen $\Theta(N^2)$ olarak yazılır. $O(2N^2)$ yazmak doğrudur ama stil olarak kullanılmaz; öndeki katsayı atılır. $4N^2 = O(N^2 + N)$ de notasyon olarak doğrudur, ama düşük dereceli N atılıp $O(N^2)$ yazılır. Amaç notasyonları sade biçimde, tek terimle göstermektir.

8.3 Limit teoremi

Hangi fonksiyonun daha hızlı büyüdüğünü anlamanın bir yolu da limittir. f ve g girdisi n olan iki fonksiyon olsun; n sonsuza giderken $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ incelenir:

- **Limit 0 ise:** f çok az büyürken g çok daha hızlı büyüyordur, kesir sıfıra yaklaşır. Bu durumda $f(n) = o(g(n))$.
- **Limit sıfırdan farklı bir sabit ise:** İkisi benzer oranda büyüyordur. $f(n) = \Theta(g(n))$ ve Theta'nın simetrisinden $g(n) = \Theta(f(n))$.
- **Limit sonsuz ise:** f, g'den çok daha hızlı büyüyordur. $g(n) = o(f(n))$.
- **Limit 0, sabit ya da sonsuz çıkmıyorsa:** İki fonksiyon sürekli birbirini geçiyordur, biri hızlı büyürken diğeri hızlı büyüyordur. Bu durumda aralarında bu bağıntılardan hiçbiri yazılamaz.

Hoca bunun, c katsayısı bulmakta zorlanıldığında ya da limit kolayca alınabildiğinde kullanılabilecek bir araç olduğunu söyledi. Slayttaki limit alıştırmalarını öğrencilere bıraktı.

9. Maksimum Alt Dizi Toplamı Problemi

9.1 Problem

Slayttaki tanım (ekrandan, 01:29:20): olası negatif tam sayılar $A_1, A_2, \ldots, A_N$ verildiğinde

$$\max_{1 \leq i \leq j \leq N} \sum_{k=i}^{j} A_k$$

bulunur. Yani komşu (ardışık) elemanlardan oluşan ve toplamı en büyük olan alt dizi aranır.

Slayttaki dizi: $-2, 11, -4, 13, -5, -2$. Hoca birkaç alt dizi denedi:

- $-2 + 11 = 9$
- $11 + (-4) + 13 = 20$; slaytta $A[2, 4] \rightarrow$ toplam $= 20$
- $13 + (-5) = 8$
- $11 + (-4) + 13 + (-5) = 15$; slaytta $A[2, 5] \rightarrow$ toplam $= 15$

Bu dizide en büyük toplam 20'dir (python ile bütün alt diziler denenerek doğrulandı).

9.2 Farklı algoritmalar

Bu problemi çözen birçok algoritma vardır: N^3, N^2, $N \log N$ ve N zamanlı. Slayttaki grafikte (ekrandan, 01:30:00) x ekseni girdi boyutu, y ekseni zamandır; $O(N^3)$, $O(N^2)$, $O(N \log N)$ ve $O(N)$ eğrileri çizilmiştir. Altındaki notlar: "küçük N için bütün algoritmalar hızlıdır" ve "büyük N için 4. algoritma (doğrusal olan) en iyisidir". Küçük N'de eğriler birbirini keser; örneğin N^2 bazı küçük değerlerde N'den ya da $N \log N$'den daha hızlı olabilir.

Hoca problemin zor görünmediğini, birkaç for döngüsüyle herkesin N^3'lük bir çözüm yazabileceğini söyledi. Kaba kuvvet (brute force) çözümde:

- Dizinin başından sonuna giden bir döngü,
- Başlangıç noktası i'yi tutup oradan ilerleyen bir döngü,
- i'den j'ye toplamı hesaplayan bir döngü

vardır. Nerede kalındığı ve o ana kadarki en büyük toplam tutulmalı, dizinin sonuna kadar gidilmelidir, çünkü sonda çok büyük bir sayı olabilir. Bu, yazılabilecek en kötü algoritmadır.

Küçük bir iyileştirme: j bir ilerleyince yeni bir eleman gelir; i'den j'ye her şeyi baştan toplamak yerine önceki toplam bir değişkende tutulup yeni eleman eklenir. Bu küçük hareketle üçüncü döngüden kurtulunur ve süre N^2'ye iner.

$N \log N$ çözümde parçaların en büyük toplamları birleştirilir: birleşim daha büyük bir toplam veriyorsa onunla, vermiyorsa diğeriyle devam edilir. Hoca bu algoritmaların ayrıntısını ilerleyen derslere bıraktı. Asıl mesaj: küçük N'de fark görülmeyebilir, ama N çok büyüyünce $N \log N$ ile N^3 arasında "dağlar kadar fark" oluşur ve kötü algoritma çok uzun süre çalışabilir.

10. Döngülerin ve if-else Yapısının Analizi

10.1 Döngüler

Table yöntemi geçen hafta görüldüğü için tekrar anlatılmadı. Asimptotik olarak:

- 0'dan n'e giden tek döngü aslında $n + 1$ kez kontrol yapar, ama $\Theta(n)$'dir. Geçen haftaki örnekte table yöntemi $2n + 3$ vermişti (1, $n + 1$, n, 1); bu da $\Theta(n)$ olarak ifade edilir.
- İç içe iki döngü m ve n kez dönüyorsa $2mn$ gibi terimlerden kurtulunur ve $\Theta(m \cdot n)$ yazılır.
- Biri Q'ya, diğeri R'ye kadar dönen iç içe iki döngü $\Theta(R \cdot Q)$'dur.

10.2 Ardışık deyimler

Slayttaki Kural 3 (ekrandan, 01:35:20): ardışık deyimlerin süreleri toplanır.

```
for ...            } θ(N)
    ...;
for ...            }
    for ...        } θ(N²)
        ...;       }
                   toplam: θ(N²)
```

Tek döngü n, iç içe iki döngü n^2 kez çalışır. İki döngü birbirinin içinde değil, art arda olduğu için toplam $\Theta(n + n^2)$ gibi yazılır. Düşük dereceli n atılınca çalışma zamanı $\Theta(n^2)$ olur.

10.3 if-else

```
if (koşul)      T3(n)
    S1          T1(n)
else
    S2          T2(n)
```

Koşula her durumda bakılır, bu yüzden T_3 her zaman eklenir; sonra T_1 ya da T_2 çalışır. Table yönteminde kesin sonuç istenirse "if'e girme olasılığı $\times T_1$ + else'e girme olasılığı $\times T_2$" yazılır. Asimptotik olarak ifade edilecekse T_1 ile T_2'den büyük olan alınabilir.

Soru: Bir dalın olasılığı düşük ama karmaşıklığı yüksekse, olasılık üzerinden mi gideceğiz, karmaşıklık üzerinden mi? **Cevap:** En doğrusu, olasılıklar biliniyorsa önce her dalı olasılığıyla çarpmak, sonra en büyüğü almaktır. Olasılıklar alınmadan doğrudan büyük dal alınırsa gereğinden yüksek tahmin (overestimate) yapılabilir: T_2'ye çok nadir giriliyor ama orada çok işlem yapılıyorsa çok daha yukarıdan bir üst sınır bulunur.

Slayttaki if-else örneği: en kötü durumda $T_3 + \max(T_1, T_2)$, en iyi durumda $T_3 + \min(T_1, T_2)$, ortalamada doğru olma (if'e girme) ve yanlış olma (else'e girme) olasılıklarıyla ağırlıklı toplam alınır. Sayısal örnekte $T_3 = n$, $T_1 = n^2$, $T_2 = n$:

- **En kötü durum:** $\Theta(n) + \max(n^2, n) = \Theta(n^2)$
- **En iyi durum:** $\Theta(n) + \min(n^2, n) = \Theta(n)$
- **Ortalama durum ($p = 1/2$):** Girdilerin yarısı if'e, yarısı else'e giriyor gibi düşünülür. $\frac{1}{2}n + \frac{1}{2}n^2$ terimlerinden yine $\Theta(n^2)$ çıkar. (Hoca T_3'ü bu toplamda ayrıca söylemedi; eklense de sonuç $\Theta(n^2)$'dir.)

Buna göre bu yapı için kabaca üst sınır $O(n^2)$, en iyi durumdan da $\Omega(n)$ yazılabilir.

11. En Kötü, Ortalama ve En İyi Durum

Slayttaki tanımlar (ekrandan, 01:40:00):

$$T_{worst}(N) = \max_{|I|=N} \{T(I)\} \quad \text{(genelde kullanılan)}$$

$$T_{av}(N) = \sum_{|I|=N} T(I) \cdot \Pr(I) \quad \text{(hesaplaması zor)}$$

$$T_{best}(N) = \min_{|I|=N} \{T(I)\}$$

$$T_{worst}(N) \geq T_{av}(N) \geq T_{best}(N)$$

$$T(n) = O(T_{worst}(n)) = \Omega(T_{best}(n))$$

Doğrusal arama (linear search) üzerinden açıklandı. En kötü durum T'nin en uzun sürdüğü durumdur: aranan eleman ya en sondadır ya da hiç yoktur. Ortalama durumda bir toplam vardır: birinci elemanda bulma olasılığı çarpı birinci elemana bakma zamanı, artı ikinci elemanda bulma olasılığı çarpı ikinci elemana bakma zamanı, … diye hepsi toplanır. En iyi durum, aranan elemanın ilk elemanda bulunmasıdır.

Doğrusal arama için en kötü durum n, olasılıklar eşitse ortalama durum yaklaşık $n/2$, en iyi durum 1'dir. Dolayısıyla $n \geq n/2 \geq 1$ ve çalışma zamanı için $O(n)$ ve $\Omega(1)$ yazılabilir.

> **Ek bilgi (derste anlatılmadı):** Eleman dizide bulunuyor ve her konumda olma olasılığı $1/n$ ise beklenen karşılaştırma sayısı $\sum_{i=1}^{n} i \cdot \frac{1}{n} = \frac{n+1}{2}$'dir. Hocanın "$n/2$" dediği değer bunun yaklaşık hâlidir; ikisi de $\Theta(n)$'dir.

Hoca doğrusal aramanın Theta ile de ifade edilebileceğini görebileceğimizi söyledi ama bunu doğrusal arama slaytına geldiğinde anlatacağını belirtti.

12. Hazır Algoritma Seçimi ve Analizin Deneysel Kontrolü

12.1 Ne zaman algoritma analizi yapılır?

Klasik algoritmalar kullanılan bir iş yerinde sıfırdan algoritma yazılmayabilir. Ama bilimsel hesaplama yapılıyorsa (örneğin atomların çarpışmasını simüle eden bir kod) ya da savunma sanayinde füzelerin hızı, sürtünme gibi pek çok etkiyi hesaplayan büyük bir kod yazılıyorsa, sıfırdan yazılan algoritmaların analizi yapılmalıdır.

Sıralama ya da permütasyon gibi hazır bir algoritma kullanılacaksa sıfırdan yazmaya uğraşmamak gerekir, çünkü kütüphanelerde daha verimli biçimde uygulanmışlardır. Ama Google'da ilk çıkanı da kullanmamak gerekir. Kendi probleminize ve girdilerinizin nasıl olduğuna bakıp algoritmanın en iyi, ortalama ve en kötü durumlarını incelemelisiniz. Örneğin bir algoritma ortalama durumda n'de, en kötü durumda n^3'te çalışıyor olabilir. Girdileriniz çoğunlukla en kötü duruma düşüyorsa, bakmadan seçilen bu algoritma çok kötü bir seçim olur.

Sıfırdan bir şey yazılıyorsa hemen koda başlanmaz. Önce algoritmanın ne kadar çalışacağına teorik olarak bakılır, sonra uygulanır (implement), sonra deneysel olarak kontrol edilir.

12.2 Deneysel kontrol

Slayttaki yöntem (ekrandan, 01:50:00):

- $T_t(N)$: teorik çalışma zamanı
- $T_e(N)$: deneysel çalışma zamanı
- $T_t(N) = O(f(N))$ olsun ve $\dfrac{T_e(N)}{T_t(N)} = \dfrac{T_e(N)}{f(N)}$ hesaplansın.
  - Oran bir sabite yakınsıyorsa $f(N)$ sıkı sınırdır.
  - Sıfıra yakınsıyorsa sınır sıkı değildir (overestimate).
  - Iraksıyorsa süre olduğundan küçük tahmin edilmiştir (underestimate).

Rastgele girdiler üretilir, boyutları $N, 2N, 4N, \ldots$ diye artırılır. Kâğıt kalemle hesaplanan teorik değer yazılır, sonra algoritma gerçek donanımda çalıştırılır. Oran sabite yakınsıyorsa teorik analiz iyi yapılmıştır, algoritma beklendiği gibi çalışır. Sıfıra yaklaşıyorsa çok yukarıdan bir sınır bulunmuştur; teorik analizde birçok şey yok sayıldığı için, karmaşık görünen bir işlem donanımda çok daha hızlı çalışıyor olabilir.

Soru: Birimleri farklıyken mukayeseyi nasıl yapacağız? **Cevap:** Deneysel süre bilgisayarın çalışma zamanı olarak, örneğin nanosaniye cinsinden verilebilir. Bakılan şey değerlerin kendisi değil, büyüme biçimidir. Elma ile armut gibi farklı birimler olsa da soru şudur: girdi iki katına çıkınca deneysel süre de iki katına mı çıkıyor, yoksa birden beş katına mı?

Süre teorik tahminden çok daha hızlı artıyorsa, yani girdi iki katına çıkınca gerçek süre çok daha fazla artıyorsa, eksik tahmin yapılmıştır. Bunun sebepleri: olasılıklar yanlış hesaplanmış olabilir ya da yok sayılan terimler N büyüdükçe çok daha önemli hâle gelmiş olabilir. Bu yüzden gerçek hayatta deneysel kontrol de yapılır; teorik analiz yapıldı diye deneye gerek kalmaz denemez. Teorik analizin katkısı, matematiksel olarak koca polinomlardan kurtulup algoritmaları "n^2'lik", "doğrusal zamanlı" gibi sınıflara ayırmayı sağlamasıdır.

Hoca slaytlardaki bir grafikte görünen mavi eğrinin bir çalışma zamanı olamayacağını belirtti: girdi arttıkça azalan bir eğridir, oysa çalışma zamanı fonksiyonunun azalmayan bir fonksiyon olduğu dersin başında söylenmişti. Bu, slayttaki bir hatadır.

13. Algoritma Karmaşıklığı ve Problem Karmaşıklığı

Şimdiye kadar Theta, Omega gibi notasyonlarla anlatılan şey algoritmanın karmaşıklığıydı. Bir de problemin karmaşıklığı vardır: bir problemin en az ne kadar zamanda çözülebileceği. Problemin üst sınırı, bilinen bir algoritmanın problemi en fazla bu kadar zamanda çözdüğünü; alt sınırı ise problemi çözecek her algoritmanın en az bu kadar zaman harcayacağını söyler.

Doğrusal aramada mantıksal olarak bütün elemanlara bir kez bakmak zorunludur. Sıralamada da kabaca her elemana bir kez bakmak gerekir; dolayısıyla sıralamanın alt sınırı n'den küçük

olamaz. Hoca sıralamada küçüktür, büyüktür gibi ek işlemler olduğunu ve teorik olarak alt sınırın n'e eşit olabileceği gibi bir ifade kurulabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Karşılaştırmaya dayalı sıralama algoritmaları için büyük olasılıkla ders kitaplarında da verilen sonuç, en kötü durumda $\Omega(n \log n)$ karşılaştırma gerektiğidir. Bunun gerekçesi, $n!$ olası sıralamayı ayırt eden bir karar ağacının yüksekliğinin en az $\log_2(n!)$ olmasıdır. Yani "her elemana bir kez bakmak" sınırı olan $\Omega(n)$ doğru ama sıkı olmayan bir alt sınırdır.

Slaytlarda geçen haftadan kalan örnekler de var: doğrusal arama, maksimum alt dizi toplamı, en büyük ortak bölen (Öklid algoritması) ve insertion sort. Hoca bunlara slaytlardan bakılmasını önerdi.

14. Özyinelemeli Çağrılar ve Yineleme Bağıntıları

14.1 Faktöriyel örneği

Slayttaki kod (ekrandan, 02:00:00):

```
int factorial(int n)
{
    if (n <= 1)
        return 1;
    else
        return n * factorial(n - 1);
}
```

n birden küçük ya da eşitse (negatif, 0 ya da 1) 1 döndürülür; yoksa fonksiyon n ile kendisini $n-1$ için çağırmasının çarpımını döndürür. Bir for döngüsü olsaydı çalışma zamanı kolayca hesaplanırdı. Özyinelemede (recursion) ise fonksiyonun kendini kaç kez çağıracağına bakmak gerekir. Bu yüzden özyinelemeli algoritmalar diğer algoritma analizlerinden farklı biçimde ele alınır.

Table yöntemiyle bakılırsa: if'e girilen durumda yalnız bir sabit döndürülür; slaytta bu kısım 0 maliyetle gösterilmiş. Else durumunda bir çarpma, bir çıkarma ve $n-1$ için faktöriyelin tekrar hesaplanması vardır. $T(n)$ faktöriyel fonksiyonunun çalışma zamanı olmak üzere:

$$T(n) = 2 + T(n-1), \qquad T(1) = 1$$

Bunun Big O'su ya da Theta'sı hemen bilinmez; yinelemenin analiz edilmesi gerekir.

14.2 İleri yerine koyma ile çözüm

$T(n-1)$ çağrıldığında aynı fonksiyona yeniden girilir: yine çarpma ve çıkarmadan 2 gelir ve bu kez $T(n-2)$ çağrılır. Bu, $n \leq 1$ olana, yani $T(1)$'e ulaşılana kadar sürer. Slayttaki açılım (ekrandan, 02:10:00):

$$T(n) = 2 + T(n-1)$$
$$T(n) = 2 + 2 + T(n-2)$$
$$T(n) = 2 + 2 + 2 + T(n-3)$$

$n-3$ satırında üç tane 2 vardır. Bir k değişkeniyle genelleştirilirse:

$$T(n) = k \cdot 2 + T(n-k)$$

$k = 1$ birinci satıra, $k = 2$ ikinci satıra uyar. $k = n-1$ yazılırsa başlangıç koşuluna gelinir:

$$T(n) = (n-1) \cdot 2 + T(n-(n-1)) = (n-1) \cdot 2 + T(1) = (n-1) \cdot 2 + 1 = 2n - 1 = \Theta(n)$$

Böylece yinelemeden kurtulunmuş olur. Hoca bu yönteme "forward substitution" dedi: elde bir yineleme denklemi vardır, ileriye doğru açılır, genel bir k terimiyle ifade edilir, sonra k'dan başlangıç durumuna gidilerek düz bir ifade bulunur.

> **Ders sonrası not:** Hocanın bu ilk örnekte "forward substitution" dediği işlem ($T(n)$'den $T(n-1)$, $T(n-2)$'ye doğru açma), aynı derste biraz sonra verilen tanıma göre geri yerine koymadır (backward substitution). İleri yerine koyma başlangıç koşulundan ($T(1)$, $T(2)$, ...) yukarı doğru gider.

14.3 Fibonacci örneği

Slayttaki kod (ekrandan, 02:14:40):

```
Fib(int N)
{
    if (N <= 1)
        return 1;
    else
        return Fib(N-1) + Fib(N-2);
}
```

Fibonacci serisinde her eleman kendinden önceki iki sayının toplamıdır. $N = 5$ gelirse fonksiyon 4 ve 3'ü çağırır; 4 de 3 ile 2'yi çağırır ve böyle devam eder. Başlangıç koşulları $T(0) = 1$ ve $T(1) = 1$'dir, çünkü ikisi de if'e girer ve bir kez çalışır. Slaytta yineleme şöyle yazılmış:

$$T(N) = T(N-1) + T(N-2) + 2 \;\Rightarrow\; T(N) = O(2^n) \quad \text{(tümevarımla)}$$

Hoca else dalında iki çıkarma ve bir toplama olduğu için $+2$ yerine $+3$ yazmanın daha mantıklı olacağını söyledi. Sonra "bu yinelemenin çalışma zamanı nedir, $\Theta(n)$ mi, $\Theta(n^2)$ mi, $\Theta(n^3)$ mü?" diye sordu ve çözümü yineleme çözme yöntemlerinin anlatıldığı ikinci slayt grubuna bıraktı. Bu sorunun bir çözümü "Örnek Sorular" bölümündedir.

14.4 Yineleme bağıntıları ve analiz adımları

İkinci slayt grubu yineleme bağıntılarının (recurrence relation) nasıl çözüldüğü üzerinedir. Görülecek yöntemler:

- İleri yerine koyma (forward substitution)
- Geri yerine koyma (backward substitution)
- Diferansiyel denklem çözmeye benzeyen ve tam çözümü (exact solution, örneğin $n^2 + 1$ gibi) veren bir yöntem
- Tam çözümü bulmadan doğrudan asimptotik sonucu veren ana teorem (master theorem)

Yineleme bağıntıları genellikle özyinelemeli algoritmalarda ortaya çıkar. Slayttaki örneklerden biri $f(n) = f(n/2) + 1$, $f(0) = 1$'dir; bir diğeri $f(n)$, $f(n-1)$ ve $6f(n-2)$ terimlerini içeren, başlangıç koşulları $f(0) = 1/4$ ve $f(1) = 2$ olan bir bağıntıdır. Hoca bu ikincisinin slayttaki çözümünün nasıl bulunacağının ileride görüleceğini söyledi.

Özyinelemeli bir algoritmanın analizi şu adımlarla yapılır:

- Girdiye bağlı parametre belirlenir.
- Temel işlem (basic operation) belirlenir: toplama mı, çarpma mı sayılacak?
- Fonksiyonun kendini nasıl çağırdığına bakılır: kendini üçe bölerek mi, iki kez mi, bir çıkararak mı çağırıyor?
- Temel işleme ve çağrılma biçimine göre yineleme bağıntısı yazılır, örneğin $T(n) = T(n/2) + 7$.
- Başlangıç koşulları (initial conditions) yazılır. Özyinelemeli bir algoritma bir yerde bir if'e girerek durmak zorundadır, yoksa sonsuz kez çağrı olur; başlangıç koşulları bu durma durumlarıdır.
- Yineleme çözülür.

14.5 Faktöriyel: çarpma sayısı

Slayttaki ikinci faktöriyel sürümü (ekrandan, 02:20:00):

```
Algorithm:
  if n = 0 return 1
  else return F(n-1) * n
```

Burada temel işlem çarpmadır:

$$M(n) = M(n-1) + 1 \quad (n > 0), \qquad M(0) = 0$$

$M(n-1)$ terimi $F(n-1)$'i hesaplamaktan, $+1$ de $F(n-1)$'i n ile çarpmaktan gelir. $n = 0$'da sabit döndürüldüğü için başlangıç değeri 0 alındı. Hoca bunun bazı durumlarda 1 de alınabileceğini, bunun bir atama işlemi sayılabileceğini, asimptotik sonucu çok değiştirmeyeceğini söyledi. Slaytın yanına da $T(0) = 1, T(n) = T(n-1)+1$ notunu ekledi. Aynı şekilde çıkarmayı da sayıp $+2$ almak sonucu çok etkilemez.

İleri yerine koyma. Başlangıç koşulundan başlanır, birkaç terim yazılır, bir örüntü çıkarılır:

$$M(0) = 0, \quad M(1) = M(0) + 1 = 1, \quad M(2) = M(1) + 1 = 2, \quad M(3) = M(2) + 1 = 3$$

Buradan $M(n) = n$ tahmin edilir. Ama bu yeterli değildir: "n dedim" diye n olduğu ispatlanmış olmaz. Bulunan kapalı form (closed form) formülün doğruluğu genellikle tümevarımla (induction) ya da yerine koyarak ispatlanmalıdır.

14.6 Geri yerine koyma

Geri yerine koymada n'den başlanır, $n-1$, $n-2$, $n-3$'e doğru gidilir, bir örüntü bulunur ve örüntü kullanılarak başlangıç koşuluna gidilip formül elde edilir. Aynı faktöriyel yinelemesi için:

$$M(n) = M(n-1)+1 = [M(n-2)+1]+1 = M(n-2)+2 = [M(n-3)+1]+2 = M(n-3)+3$$

Yapılan şey, $M(n-1)$'in formüldeki karşılığını denklemde yerine koymak, sonra $M(n-2)$'nin karşılığını koymak ve bunu sürdürmektir. -3 ile $+3$ birlikte göründüğü için genel terim

$$M(n) = M(n-i) + i$$

olarak yazılır. Başlangıç koşulu $M(0) = 0$'dır. $i = n$ seçilince $M(0)$'a ulaşılır ve $M(n) = 0 + n = n$ bulunur; bu da $\Theta(n)$'dir.

Soru: Orada i yerine n koyarak sıfırı elde etmiş olmadık mı? **Cevap:** Evet, $n - n = 0$ oldu ve M terimi $M(0)$'a dönüştü, geriye $+n$ kaldı. Önemli olan i'nin neden n seçildiğidir: $M(0)$'ın değeri bilindiği için başlangıç koşulu yerine konarak M'li ifadeden kurtulmaya çalışılır.

İlk faktöriyel sürümünde $+2$ alındığı için genel terim $k \cdot 2 + T(n-k)$ çıkmıştı, ikinci sürümde $M(n-i)+i$. Birincinin cevabı $2n-1$, ikincininki n'dir. Theta yazılırken öndeki katsayı ve sabitler atıldığı için bir çarpma mı yoksa bir çarpma ile bir çıkarma mı sayıldığı sonucu etkilemez; ikisi de $\Theta(n)$'dir. İki örnek de geri yerine koymaya örnektir.

15. Hanoi Kuleleri ve Kök İçeren Bir Yineleme

15.1 Problem

Hoca problemi bir animasyonla gösterdi. Başlangıçta bütün diskler A çubuğunda büyükten küçüğe dizilidir, en küçük en üsttedir. Amaç hepsini aynı düzende C çubuğuna taşımaktır. Kural: küçük disk her zaman büyük diskin üstüne konabilir, büyük disk küçük bir diskin üstünde olamaz. Hareket ederken üçüncü çubuk yardımcı olarak kullanılır.

Üç diskli örnekte (en büyük yeşil, ortanca beyaz, en küçük kırmızı): kırmızı C'ye, beyaz B'ye, kırmızı B'ye konur; sonra yeşil A'dan C'ye taşınır; sonra beyaz ve kırmızı uygun sırayla C'ye aktarılır.

15.2 Özyinelemeli çözüm ve yineleme

Slayttaki yaklaşım (ekrandan, 02:30:00): n diski A'dan C'ye B'yi kullanarak taşımak için

- $n-1$ diski A'dan B'ye C'yi kullanarak taşı,
- en büyük diski A'dan C'ye taşı,
- $n-1$ diski B'den C'ye A'yı kullanarak taşı.

Algoritmada iki kez $n-1$ disk taşıma işlemi vardır (birinci ve üçüncü adım), bir de en büyük diskin taşınmasından gelen 1 hamle. Tek disk varsa yalnız ikinci adım yapılır, disk doğrudan A'dan C'ye konur. Toplam hamle sayısı:

$$T(n) = 2T(n-1) + 1, \qquad T(1) = 1$$

15.3 Geri yerine koyma ile çözüm

Slayttaki çözüm (ekrandan, 02:33:00):

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ T(n) &= 2(2T(n-2)+1) + 1 = 4T(n-2) + 2 + 1 \\ T(n) &= 4(2T(n-3)+1) + 2 + 1 = 8T(n-3) + 4 + 2 + 1 \\ &\vdots \\ T(n) &= 2^i T(n-i) + 2^{i-1} + 2^{i-2} + \cdots + 2^1 + 1 \end{aligned}$$

Hoca slayttaki bir satırda yazım hatası olduğunu söyledi, sonra satırın yerine koyma değil, bir önceki satırın açılmış hâli olduğunu fark etti. Örüntü şöyle okunur: $8 = 2^3$ ve yanında $T(n-3)$ vardır; toplam terimleri $2^2 + 2^1 + 2^0$'dır. Dolayısıyla 8 yerine 2^i, 3 yerine i, toplam yerine $2^{i-1} + 2^{i-2} + \cdots + 2^0$ yazılır. Slaytta bu genel terimin yanında "ispatlanması gerekir" notu vardır.

Bu formülü yazmanın amacı T'li kısımdan, yani fonksiyonun kendini çağırdığı kısımdan kurtulmaktır. Bunun için başlangıç koşulu $T(1) = 1$ kullanılır: $T(n-i) = T(1)$ olacak i aranır, yani $i = n-1$:

$$T(n) = 2^{n-1}T(1) + 2^{n-2} + \cdots + 2^1 + 1 = 2^{n-1} + 2^{n-2} + \cdots + 1 = 2^n - 1 = \Theta(2^n)$$

Disk sayısı büyüdükçe çalışma zamanı üstel olarak artar. $n = 1, \ldots, 7$ için yineleme ile $2^n - 1$ python ile karşılaştırıldı, değerler aynıdır.

Hoca yöntemi şöyle özetledi: ilk adımlarda adım adım açılarak yazılır; birkaç adımdan sonra bunun genel bir formülle ifade edilip edilemeyeceğine bakılır; genel formülde i yerine öyle bir n'li terim konur ki başlangıç durumu elde edilsin. Böylece i'lerden kurtulup içinde yalnız n olan bir çalışma zamanı bulunur.

Soru: Geri yerine koyma ile ileri yerine koyma arasındaki farkı kaçırdım. **Cevap:** Geri yerine koymada n'den başlanır, $n-1$, $n-2$, $n-3$ üzerinden başlangıç durumu $T(1)$'e gidilir. İleri yerine koymada $T(1) = 1$'den başlanır, $T(2) = 2T(1) + 1$ diye 1'den 2'ye, 3'e doğru ileri gidilir. İkisinde de bir formül tahmin edilir.

Soru: Bu problem NP sınıfı çıktı herhalde, değil mi? **Cevap:** Hoca evet dedi, 2^n ile çıktığını söyledi. Bu algoritma 15 ya da 100 diskle yazılıp çalıştırılırsa bugünkü bilgisayarlarda büyük olasılıkla sonlanmayacağını, 3 disk gibi az sayıda diskle ise elbette sonlanacağını ekledi.

Ders sonrası not: Hanoi probleminin süresi üsteldir, ama bu büyük olasılıkla NP kavramıyla karıştırılmıştır. NP, çözümü polinom zamanda doğrulanabilen karar problemlerinin sınıfıdır; Hanoi'de üstel sürenin sebebi çıktının kendisinin $2^n - 1$ hamle olmasıdır. Ayrıca 15 disk $2^{15} - 1 = 32767$ hamle eder ve bugünkü bir bilgisayarda hemen biter; sonlanmama sorunu 100 gibi büyük disk sayılarında ortaya çıkar (2^{100} mertebesinde hamle).

15.4 Kök içeren yineleme

Slayttaki örnek (ekrandan, 02:40:00):

$$T(n) = 2T(\sqrt{n}) + 1, \qquad T(2) = 0$$

Bu da geri yerine koymayla çözülür. $\sqrt{n}$ yerine $n^{1/2}$ yazılır ve ifade kendi içinde yerine konur. n yerine $n^{1/2}$ yazılınca $n^{1/2 \cdot 1/2} = n^{1/4}$ olur:

$$T(n) = 2T(n^{1/2}) + 1$$
$$T(n) = 2(2T(n^{1/4}) + 1) + 1 = 4T(n^{1/4}) + 1 + 2$$
$$T(n) = 8T(n^{1/8}) + 1 + 2 + 4$$
$$\vdots$$
$$T(n) = 2^i T(n^{1/2^i}) + 2^0 + 2^1 + \cdots + 2^{i-1}$$

Başlangıç koşulu $T(2)$ olduğu için $n^{1/2^i}$'nin 2'ye eşit olması gerekir. i yerine hangi terim yazılmalı? Taban 2 olmak üzere $i = \log\log n$ yazılırsa:

$$n^{1/2^i} = 2 \;\Rightarrow\; 2^i = \log n \;\Rightarrow\; i = \log\log n \quad \text{(ara adım eklendi)}$$

Bu durumda ilk terim $2^{\log\log n} \cdot T(2) = 2^{\log\log n} \cdot 0 = 0$ olur ve sıfırlanır. Geriye

$$T(n) = 2^0 + 2^1 + \cdots + 2^{\log\log n - 1}$$

kalır. Bu toplam, bir önceki Hanoi slaytındaki $2^n - 1$ durumuna benzer. Toplam $2^{\log\log n} - 1 = \log n - 1$ olur (ara adım eklendi), yani $\Theta(\log n)$. Python ile $n = 2^{2^k}$ ($k = 1, \ldots, 5$) için yineleme hesaplandı, sonuç $\log n - 1$ çıktı.

Soru: Burada logaritmanın tabanı 2 olduğu için $2^{\log}$ ifadesi sadeleşiyor ve $\log n$'e dönüşüyor, değil mi? **Cevap:** Evet, aynen öyle.

Hoca bu örneğin önemini şöyle açıkladı: köklü bir ifade varsa kökten kurtulmak için genellikle logaritmalı ifadeler kullanılır. Özellikle ağaçlarda logaritmik zamanda çalışan durumlarda, i'yi içinde n olan bir terim olarak yazmaya çalışırken kullanılan şey genellikle logaritma olur.

Örnek Sorular

Soru 1 (ekrandan, 00:05:20): Girdi boyutu N için A algoritmasının çalışma zamanı $T_A(N) = 1000N$, B algoritmasınınki $T_B(N) = N^2$ ise hangisi daha hızlıdır?

Çözüm (derste verildi): Girdi boyutu bilinmeden kesin bir şey söylenemez. $N < 1000$ için $N^2 < 1000N$ olduğundan B daha hızlıdır; $N = 1000$'de ikisi eşittir; $N > 1000$ için A daha hızlıdır. Asimptotik olarak $T_A = O(N)$, $T_B = O(N^2)$ olduğundan A, B'den asimptotik olarak daha hızlıdır.

Soru 2 (ekrandan, 00:20:40): Aşağıdakilerin doğru olup olmadığını uygun c ve n_0 ile gösterin. a) $7n + 5 = O(n)$ b) $7n + 5 = O(n^2)$ c) $7n^2 + 3n = O(n)$

Çözüm (derste verildi): a) Doğru. $c = 8$, $n_0 = 5$: $n \geq 5$ için $7n + 5 \leq 8n$. b) Doğru. $c = 7$, $n_0 = 2$: $n \geq 2$ için $7n + 5 \leq 7n^2$ ($n = 2$'de $19 \leq 28$). c) Yanlış. $n = 1$'den $c \geq 10$ gerekir; $c = 10$ için $n = 2$'de $34 \leq 20$ sağlanmaz. c ne seçilirse seçilsin $7n^2 + 3n \leq cn$, yani $7n + 3 \leq c$, büyük n'de bozulur.

Soru 3 (derste sözlü, 00:42:30): $4n^3$'ü $O(n^3)$ olarak yazarken c kaç alınmalıdır?

Çözüm (derste verildi): Polinomda başka terim yoksa $c = 4$ yeter: $4n^3 \leq 4n^3$. $c = 3$ yetmez. Polinomda düşük dereceli başka terimler varsa c onları da karşılayacak kadar büyük seçilmelidir.

Soru 4 (ekrandan, 00:50:00): Aşağıdakileri gösterin. a) $3n + 2 = \Omega(n)$ b) $6 \cdot 2^n + n^2 = \Omega(2^n)$ c) $3n - 7 = \Omega(1)$ doğru mudur, iyi bir alt sınır mıdır?

Çözüm: a) Derste $3n + 2 \geq cn$ biçiminde kuruldu. Hocanın söylediği $c = 4$ büyük n için sağlanmaz (bkz. bölüm 5.2'deki ders sonrası not); $c = 3$, $n_0 = 1$ ile her $n \geq 1$ için $3n + 2 \geq 3n$. c) Derste verildi: doğrudur, çünkü sabit fonksiyon artan $3n - 7$'nin (yeterince büyük n için) alt sınırıdır; ama sıkı olmadığı için iyi bir alt sınır değildir.

> **Çözüm (derste verilmedi):** b) $n^2 \geq 0$ olduğundan her $n \geq 1$ için $6 \cdot 2^n + n^2 \geq 6 \cdot 2^n$. Dolayısıyla $c = 6$, $n_0 = 1$ ile $6 \cdot 2^n + n^2 = \Omega(2^n)$ (python ile $n = 1, \ldots, 49$ için de denetlendi).

Soru 5 (derste sözlü, 00:57:13): $7n^2 + 3n + 5 = \Omega(n)$ nasıl yazılabilir?

Çözüm (derste verildi): $7n^2 + 3n + 5 \geq c \cdot n$ sağlayan değerler aranır. $c = 8$ alınırsa $n = 1$'de $15 \geq 8$, $n = 2$'de $39 \geq 16$, $n = 3$'te $77 \geq 24$ ve sonrasında da sağlanır, çünkü sol taraf daha hızlı büyür. Dolayısıyla $\Omega(n)$ yazılabilir.

Soru 6 (ekrandan, 01:18:10): $T_1(N) = O(f(N))$ ve $T_2(N) = O(g(N))$ ise $T_1 + T_2 = \max(O(f), O(g))$ ve $T_1 \cdot T_2 = O(f \cdot g)$ olduğunu ispatlayabilir misiniz? Aynı kural Theta ve Omega için de geçerli midir?

Çözüm (derste kısmen verildi): Hoca polinom sezgisini verdi: toplamada aynı dereceli terimlerin katsayıları toplanır ve en yüksek derece değişmez; çarpmada üsler toplanır.

> **Çözüm (derste verilmedi):** $N \geq n_1$ için $T_1 \leq c_1 f$, $N \geq n_2$ için $T_2 \leq c_2 g$ olsun. $N \geq \max(n_1, n_2)$ için $T_1 + T_2 \leq c_1 f + c_2 g \leq (c_1 + c_2) \max(f, g)$, yani $T_1 + T_2 = O(\max(f, g))$. Çalışma zamanları pozitif olduğu için $T_1 T_2 \leq c_1 c_2 \, fg$, yani $T_1 T_2 = O(fg)$. Omega için eşitsizlikler ters yönde kurulur: $T_1 \geq c_1 f$, $T_2 \geq c_2 g$ ise $T_1 + T_2 \geq \max(c_1 f, c_2 g) \geq \min(c_1, c_2) \max(f, g)$ ve $T_1 T_2 \geq c_1 c_2 \, fg$; iki kural da Omega için geçerlidir. Theta hem O hem Omega demek olduğu için kurallar Theta için de geçerlidir.

Soru 7 (ekrandan, 01:29:20): $-2, 11, -4, 13, -5, -2$ dizisinde ardışık elemanlardan oluşan en büyük toplamlı alt dizi hangisidir?

Çözüm (derste verildi): $A[2..4] = 11 - 4 + 13 = 20$. Karşılaştırma için $A[2..5] = 15$, $-2 + 11 = 9$. Bütün alt diziler python ile denendiğinde en büyük toplam 20 çıkar.

Soru 8 (derste sözlü, 02:15:35): Fibonacci algoritması için $T(N) = T(N-1) + T(N-2) + 3$, $T(0) = T(1) = 1$ yinelemesinin çalışma zamanı nedir? (Slaytta $+2$ ile yazılmış ve sonuç $O(2^n)$, "tümevarımla" denmiştir.)

Çözüm (derste verilmedi): Slayttaki sonuç $O(2^n)$'dir; derste ispatı yapılmadı.

> **Çözüm (derste verilmedi):** $T(N) \leq 4 \cdot 2^N - 3$ olduğu tümevarımla gösterilir. Taban: $T(0) = 1 \leq 4 - 3 = 1$, $T(1) = 1 \leq 8 - 3 = 5$. Adım: $T(N) \leq (4 \cdot 2^{N-1} - 3) + (4 \cdot 2^{N-2} - 3) + 3 = 4 \cdot \frac{3}{4} \cdot 2^N - 3 = 3 \cdot 2^N - 3 \leq 4 \cdot 2^N - 3$. Dolayısıyla $T(N) = O(2^N)$. Eşitsizlik python ile $N = 0, \ldots, 24$ için de denetlendi.

Soru 9 (ekrandan, 02:33:00): Hanoi kuleleri için $T(n) = 2T(n-1) + 1$, $T(1) = 1$ yinelemesini geri yerine koymayla çözün.

Çözüm (derste verildi): $T(n) = 2^i T(n-i) + 2^{i-1} + \cdots + 2 + 1$. $i = n - 1$ seçilince $T(n) = 2^{n-1} + 2^{n-2} + \cdots + 1 = 2^n - 1 = \Theta(2^n)$.

Soru 10 (ekrandan, 02:40:00): $T(n) = 2T(\sqrt{n}) + 1$, $T(2) = 0$ yinelemesini çözün.

Çözüm (derste verildi): $T(n) = 2^i T(n^{1/2^i}) + 2^0 + \cdots + 2^{i-1}$. $n^{1/2^i} = 2$ için $i = \log\log n$. İlk terim $T(2) = 0$ yüzünden sıfırlanır; $T(n) = 2^{\log\log n} - 1 = \log n - 1 = \Theta(\log n)$.

Dersin Toparlaması

Bu derste algoritmaların çalışma zamanlarının makineden bağımsız olarak nasıl karşılaştırılacağı ele alındı. $1000N$ ile N^2 örneği, girdi boyutu bilinmeden hangi algoritmanın hızlı olduğunun söylenemeyeceğini gösterdi; bu yüzden büyük N davranışına bakan asimptotik notasyonlar tanımlandı. Big O üst sınırı, Omega alt sınırı, Theta ikisinin aynı olduğu sıkı sınırı, little o ise sıkı olmayan üst sınırı verir. Bu notasyonlarda katsayılar ve düşük dereceli terimler atılır, her zaman en sıkı sınır aranır. Toplama ve çarpma kuralları, polinomların derecesi, yaygın fonksiyonların büyüme sırası ve limit yöntemi bu karşılaştırmaların araçlarıdır. Maksimum alt dizi toplamı problemi, aynı problemi N^3'ten N'ye kadar farklı sürelerde çözen algoritmalar olabileceğini gösterdi. Döngülerde, ardışık bloklarda ve if-else yapılarında en kötü, ortalama ve en iyi durum analizinin nasıl yapılacağı, analizin deneyle nasıl kontrol edileceği de anlatıldı.

Dersin son bölümünde özyinelemeli algoritmalara geçildi. Faktöriyel, Fibonacci ve Hanoi kuleleri için yineleme bağıntıları yazıldı; ileri ve geri yerine koyma yöntemleriyle faktöriyelin $\Theta(n)$, Hanoi'nin $\Theta(2^n)$, $2T(\sqrt{n}) + 1$ yinelemesinin $\Theta(\log n)$ olduğu bulundu. Hoca sonraki derste yineleme çözümüne örneklerle ve ana teorem (master theorem) ile devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.